



Universidad de Granada

DOBLE GRADO EN INGENIERÍA INFORMÁTICA Y
MATEMÁTICAS

METAHEURÍSTICAS

Trabajo final:
Sine Cosine Algorithm (SCA)

Autor: Jesús Muñoz Velasco

Curso 2025-2026

Índice

I Descripción	2
1. Introducción y contexto	2
2. Fundamentos matemáticos	2
2.1. Ecuaciones de actualización de posición	2
2.2. Papel de cada parámetro	3
2.2.1. El parámetro r_1 : transición exploración/explotación	3
2.2.2. El parámetro r_2 : dirección del movimiento	4
2.2.3. Los parámetros r_3 y r_4	4
2.3. Geometría de la búsqueda	5
3. Descripción del algoritmo	6
3.1. Pseudocódigo	6
3.2. Complejidad computacional	6
4. Por qué funciona el SCA	6
5. Ventajas e inconvenientes	7
6. Cuándo funciona bien	7
7. Aplicaciones en casos reales	8
8. Variantes y extensiones	9
9. Conclusiones	9

Parte I

Descripción

1. Introducción y contexto

El Sine Cosine Algorithm (SCA) es una metaheurística poblacional propuesta por Seyedali Mirjalili en 2016 en el artículo “*SCA: A Sine Cosine Algorithm for solving optimization problems*” (Knowledge-Based Systems, vol. 96, pp. 120–133). Su propuesta se enmarca dentro de la familia de algoritmos basados en modelos matemáticos, también denominados *math-inspired*, en contraposición a los inspirados en fenómenos biológicos o físicos.

El SCA parte de la idea de que las funciones seno y coseno poseen propiedades geométricas naturalmente útiles para la búsqueda: oscilan de forma periódica, pueden tomar valores superiores o inferiores al rango $[-1, 1]$ si se escalan convenientemente, y su comportamiento alternante facilita tanto la exploración global del espacio de búsqueda como la explotación local alrededor de soluciones prometedoras. Desde su publicación, el algoritmo ha atraído una atención considerable en la comunidad investigadora, siendo aplicado en campos tan diversos como la ingeniería eléctrica, el aprendizaje automático, el diagnóstico médico y la planificación de rutas.

2. Fundamentos matemáticos

2.1. Ecuaciones de actualización de posición

Cada candidato a solución (denominado *agente de búsqueda*) se representa como un vector $\mathbf{X}^t \in \mathbb{R}^d$, donde d es la dimensión del problema y t el instante de iteración. En cada paso, el agente actualiza su posición dimensión a dimensión según:

$$X_i^{t+1} = \begin{cases} X_i^t + r_1 \cdot \sin(r_2) \cdot |r_3 P_i^t - X_i^t|, & \text{si } r_4 < 0,5 \\ X_i^t + r_1 \cdot \cos(r_2) \cdot |r_3 P_i^t - X_i^t|, & \text{si } r_4 \geq 0,5 \end{cases} \quad (1)$$

La ecuación puede factorizarse para visualizarla mejor. Definiendo el *desplazamiento escalado* $\Delta_i^t = r_3 P_i^t - X_i^t$ (distancia ponderada al destino, con signo) y el *factor trigonométrico* $\phi = r_1 \cdot f(r_2)$ donde $f \in \{\sin, \cos\}$, la actualización se reduce a:

$$X_i^{t+1} = X_i^t + \phi \cdot |\Delta_i^t| \quad (2)$$

El valor absoluto $|\Delta_i^t|$ garantiza que la magnitud del paso sea siempre positiva; el signo del movimiento queda determinado enteramente por ϕ : si $\phi > 0$ el agente avanza hacia el destino (o más allá), si $\phi < 0$ retrocede. Esta separación entre magnitud y dirección es lo que confiere al SCA su capacidad para cubrir simétricamente el espacio alrededor del punto destino.

Los cuatro parámetros tienen roles bien diferenciados:

Param.	Rango	Rol en la actualización
r_1	$[0, a]$ decreciente	Amplitud del movimiento (exploración/explotación)
r_2	$[0, 2\pi]$ aleatorio	Ángulo trigonométrico (dirección y signo del paso)
r_3	$[0, 2]$ aleatorio	Peso estocástico de la distancia al destino
r_4	$[0, 1]$ aleatorio	Selección equiprobable de seno o coseno

2.2. Papel de cada parámetro

2.2.1. El parámetro r_1 : transición exploración/explotación

r_1 es el parámetro de mayor importancia. Viene dado por¹

$$r_1(t) = a \left(1 - \frac{t}{T} \right), \quad a = 2 \quad (3)$$

Como se puede ver comienza con el valor a ($r_1(0) = a$) y va descendiendo linealmente hasta 0 ($r_1(T) = 0$) a lo largo de las iteraciones.

La Figura 1 muestra su trayectoria y el umbral $r_1 = 1$ que separa las dos regiones:

- **Exploración** ($r_1 \geq 1$, primeras iteraciones): el factor $r_1 \cdot f(r_2)$ puede superar en módulo la distancia $|\Delta_i^t|$, de modo que el agente sobrepasa el punto destino y explora regiones lejanas del espacio.
- **Explotación** ($r_1 < 1$): el factor queda acotado y el agente se mueve dentro del segmento que lo une con el destino, refinando la solución progresivamente.

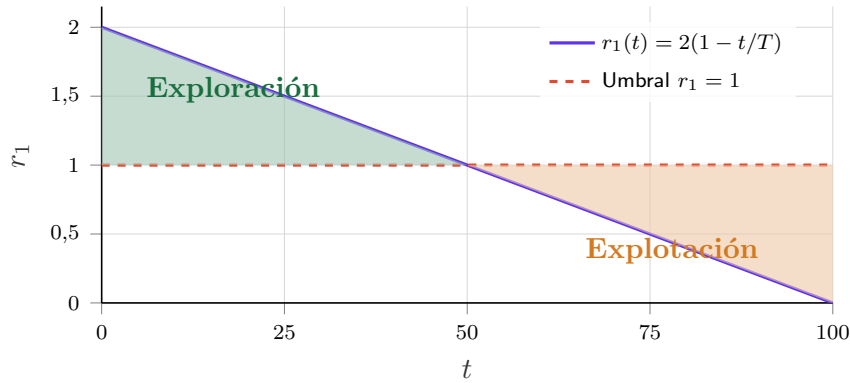


Figura 1: Evolución de r_1 a lo largo de las iteraciones ($a = 2$, $T = 100$). La región azul corresponde a la fase de exploración ($r_1 \geq 1$) y la naranja a la de explotación ($r_1 < 1$). El cruce ocurre exactamente en $t = T/2$.

¹El parámetro a se toma igual a 2 para que el espacio de búsqueda alrededor del punto destino quede completamente cubierto desde el principio (está acotado por el doble de la distancia inicial), además de que con este valor, el umbral $r_1 = 1$ se alcanza exactamente en $t = T/2$ dividiendo las iteraciones perfectamente por la mitad para exploración y explotación. Aún así se puede tomar otro valor para darle más relevancia a alguna de estas fases.

2.2.2. El parámetro r_2 : dirección del movimiento

r_2 es el ángulo de la función trigonométrica, muestreado uniformemente en $[0, 2\pi]$. Su efecto se aprecia analizando el signo de $\sin(r_2)$ y $\cos(r_2)$:

-) Cuando $f(r_2) > 0$ el desplazamiento es positivo: el agente se mueve en la dirección de $P_i - X_i$ (hacia el destino o más allá).
-) Cuando $f(r_2) < 0$ el desplazamiento es negativo: el agente se aleja del destino en esa dimensión.

La Figura 2 representa $r_1 \cdot \sin(r_2)$ y $r_1 \cdot \cos(r_2)$ para dos valores representativos de r_1 , mostrando cómo la amplitud se contrae al pasar de la fase exploratoria a la explotadora.

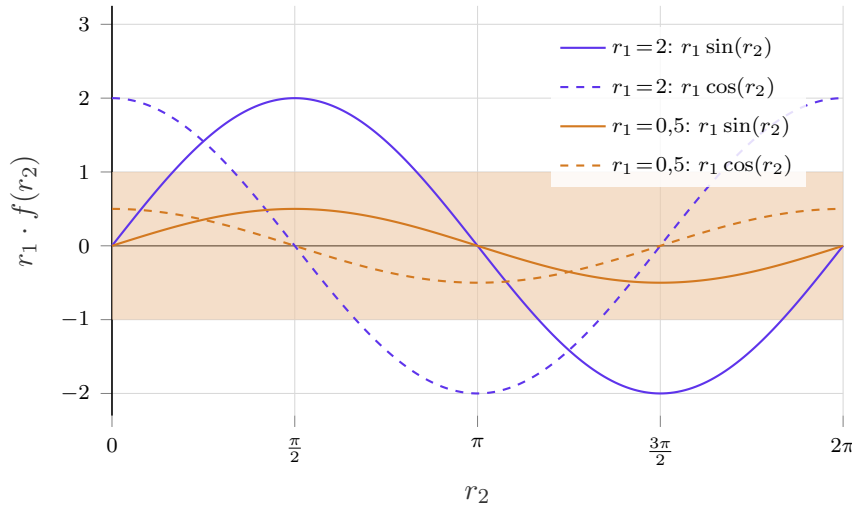


Figura 2: Factor trigonométrico $r_1 \cdot f(r_2)$ en función del ángulo r_2 para $r_1 = 2$ (exploración, azul) y $r_1 = 0,5$ (explotación, naranja). La banda sombreada $[-1, 1]$ delimita el régimen de explotación: cuando el factor queda dentro de ella, el agente no sobrepasa el destino.

2.2.3. Los parámetros r_3 y r_4

$r_3 \in [0, 2]$ actúa como un peso estocástico sobre el punto destino. Si $r_3 > 1$, el término $r_3 P_i^t$ desplaza el destino efectivo más allá de su posición real, ampliando artificialmente la distancia $|\Delta_i^t|$ y favoreciendo pasos más largos (efecto exploratorio secundario). Si $r_3 < 1$, atenúa la influencia del destino. Este mecanismo introduce diversidad adicional incluso en la fase de explotación, reduciendo el riesgo de convergencia en masa hacia un único punto.

$r_4 \in [0, 1]$ selecciona con probabilidad $1/2$ entre la función seno y la función coseno. Ambas funciones tienen la misma amplitud máxima, pero sus ceros y máximos están desfasados $\pi/2$ radianes. Emplearlas con igual probabilidad garantiza que el muestreo de direcciones de movimiento sea más uniforme que si se usase una sola función, mejorando la isotropía de la búsqueda en el espacio.

2.3. Geometría de la búsqueda

La ecuación (2) actualiza cada dimensión de forma independiente. En una dimensión fija j , el nuevo valor del agente es:

$$X_j^{t+1} = X_j^t + \phi \cdot |\Delta_j^t|, \quad |\Delta_j^t| = |r_3 P_j^t - X_j^t|, \quad \phi = r_1 \cdot f(r_2)$$

donde $|\Delta_j^t| \geq 0$ siempre. Por tanto, el signo y la magnitud del desplazamiento quedan determinados exclusivamente por ϕ , que puede tomar cualquier valor real en $[-r_1, r_1]$. Esto da lugar a cuatro regímenes de movimiento, representados en la Figura 3:

- $\phi \in (0, 1)$: explotación positiva. El agente se desplaza hacia P_j^t sin alcanzarlo.
- $\phi \in (-1, 0)$: explotación negativa. El agente retrocede levemente desde P_j^t , pero permanece cerca.
- $\phi > 1$: exploración positiva. El agente sobrepasa P_j^t y aterriza más allá.
- $\phi < -1$: exploración negativa. El agente da un salto largo en sentido opuesto a P_j^t .

Como ya se ha comentado, los casos $|\phi| < 1$ constituyen la zona de explotación y los casos $|\phi| > 1$ la zona de exploración. El umbral es exactamente $|\phi| = 1$, que corresponde al punto P_j^t escalado por r_3 . Nótese que las zonas de exploración son simétricas a ambos lados del agente, lo que garantiza que la búsqueda no tenga sesgo de dirección.

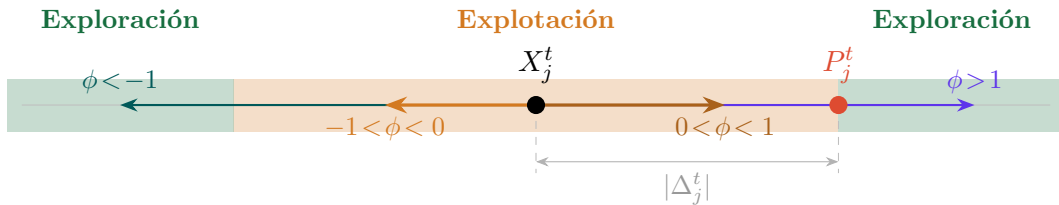


Figura 3: Cuatro regímenes de movimiento en una dimensión j . La banda amarilla ($|\phi| < 1$) es la zona de explotación: el agente aterriza entre X_j^t y el destino (o su simétrico). Las bandas azul y verde ($|\phi| > 1$) son zonas de exploración: el agente sobrepasa el destino en alguno de los dos sentidos. La simetría horizontal garantiza la ausencia de sesgo direccional.

Desde una perspectiva probabilística, el valor esperado de $|\phi|$ decrece con t al hacerlo r_1 , ya que

$$E[|\phi|] = r_1 \cdot E[|f(r_2)|] = r_1 \cdot \frac{2}{\pi}$$

Esto convierte la dinámica colectiva en un proceso de enfriamiento implícito: al inicio los agentes realizan saltos amplios y divergentes conforme avanza la búsqueda los pasos se acortan y la población converge hacia el punto destino con precisión creciente. El factor $2/\pi \approx 0,637$ implica además que, en promedio, el umbral efectivo de exploración/explotación se alcanza antes de que r_1 llegue a 1, lo que supone una transición algo más temprana de lo que la ecuación (3) sugiere a simple vista.

3. Descripción del algoritmo

3.1. Pseudocódigo

```

1  (* Parámetros de entrada: N (población), T (iteraciones máx.), a=2 *)
2  Inicializar población X = {X_1, X_2, ..., X_N} aleatoriamente en [lb, ub]^d
3  Evaluar fitness f(X_i) para cada agente i
4  P <- argmin f(X_i)      (* mejor solución: punto destino *)
5
6  t <- 1
7  Mientras t <= T hacer
8      (* Calcular parámetro adaptativo r1 *)
9      r1 <- a - t * (a / T)
10
11     Para cada agente i = 1 hasta N hacer
12         Para cada dimensión j = 1 hasta d hacer
13             (* Generar parámetros aleatorios *)
14             r2 <- aleatorio en [0, 2*pi]
15             r3 <- aleatorio en [0, 2]
16             r4 <- aleatorio en [0, 1]
17
18             (* Actualizar posición según Ec. (1) *)
19             Si r4 < 0.5 entonces
20                 X_i[j] <- X_i[j] + r1 * sin(r2) * |r3*P[j] - X_i[j]|
21             Si_no
22                 X_i[j] <- X_i[j] + r1 * cos(r2) * |r3*P[j] - X_i[j]|
23             Fin Si
24
25             (* Aplicar límites del espacio de búsqueda *)
26             X_i[j] <- max(lb[j], min(ub[j], X_i[j]))
27         Fin Para
28     Fin Para
29
30     (* Evaluar fitness y actualizar punto destino *)
31     Para cada agente i = 1 hasta N hacer
32         Evaluar f(X_i)
33         Si f(X_i) < f(P) entonces
34             P <- X_i
35         Fin Si
36     Fin Para
37
38     t <- t + 1
39 Fin Mientras
40
41 Retornar P      (* mejor solución encontrada *)

```

Código Fuente 1: Pseudocódigo del Sine Cosine Algorithm (SCA)

3.2. Complejidad computacional

La complejidad por iteración es $\mathcal{O}(N \cdot d)$, equivalente a la de otros algoritmos poblacionales estándar como PSO o GWO. El número total de evaluaciones de la función objetivo es $N \times T$, lo que resulta razonable para problemas de mediana escala.

4. Por qué funciona el SCA

El éxito del SCA descansa en tres pilares fundamentales:

1. Balance exploración–explotación adaptativo. La reducción lineal de r_1 de a a 0 a lo largo de las iteraciones crea de forma natural una transición suave de la fase exploratoria (valores grandes de r_1 permiten saltos amplios) a la explotadora

(valores pequeños de r_1 confinan el movimiento). Este mecanismo automático evita que el diseñador tenga que ajustar empíricamente la transición entre fases.

2. Diversidad mediante aleatoriedad estructurada. Los parámetros r_2 , r_3 y r_4 introducen aleatoriedad en la dirección, magnitud y función trigonométrica empleada, respectivamente. Esta triple fuente de variabilidad mantiene la diversidad de la población durante la búsqueda y reduce la probabilidad de convergencia prematura.

3. Oscilación periódica como mecanismo de escape. Las propiedades oscilantes del seno y el coseno permiten que los agentes alteren su dirección de movimiento de manera natural. En particular, en las iteraciones iniciales los valores fuera del rango $[-1, 1]$ actúan como operadores de perturbación implícitos, similares a las mutaciones en los algoritmos evolutivos, facilitando la salida de óptimos locales.

5. Ventajas e inconvenientes

Ventajas	Inconvenientes
Implementación simple: pocos parámetros (a , N , T).	Tendencia a la convergencia prematura en funciones multimodales complejas de alta dimensión.
Buen equilibrio exploración/explotación gracias al decaimiento de r_1.	La actualización lineal de r_1 no se adapta a la forma del paisaje de aptitud; puede ser subóptima en problemas no lineales.
Convergencia competitiva frente a PSO, GA y GWO en benchmarks estándar.	Baja diversidad en iteraciones avanzadas, lo que favorece el estancamiento en óptimos locales.
Bajo costo computacional por iteración: $\mathcal{O}(N \cdot d)$.	Escasa memoria colectiva: los agentes no comparten información entre ellos más allá del punto destino global.
Flexibilidad: fácilmente híbridable con otros algoritmos (PSO, DE, ABC).	Rendimiento degradado en espacios de búsqueda discretos o combinatorios sin adaptaciones adicionales.

Tabla 1: Resumen de ventajas e inconvenientes del SCA.

6. Cuándo funciona bien

El SCA muestra su mejor rendimiento en los siguientes contextos:

- **Problemas continuos de dimensión baja o media** ($d \leq 100$): la oscilación controlada por r_1 es suficiente para cubrir el espacio de búsqueda sin necesidad de operadores adicionales.

-) **Funciones unimodales o con pocos óptimos locales:** la convergencia progresiva resulta adecuada cuando el paisaje de aptitud no presenta muchas trampas locales.
-) **Situaciones con restricciones presupuestarias de evaluación:** su bajo overhead computacional lo hace conveniente cuando la evaluación de la función objetivo es costosa y el número de iteraciones debe minimizarse.
-) **Problemas de ingeniería clásicos con restricciones:** diseño de estructuras, optimización de parámetros de máquinas, flujo de potencia óptimo, donde la superficie de aptitud es relativamente suave.

Por el contrario, el SCA original tiene dificultades en problemas combinatorios puros, funciones con numerosos óptimos locales (CEC2017 con alta dimensionalidad) o cuando se requiere una precisión de convergencia muy elevada.

7. Aplicaciones en casos reales

Desde 2016, el SCA ha sido adaptado y aplicado en una amplia variedad de dominios:

Sistemas de potencia y energía. El SCA ha sido empleado para resolver el problema de *Optimal Power Flow* (OPF), minimizando pérdidas en redes eléctricas y optimizando la ubicación de compensadores estáticos de VAR (D-STATCOM) en sistemas de distribución. También se ha aplicado en la calendarización hidrotérmica a corto plazo.

Selección de características y aprendizaje automático. Se han propuesto versiones binarias del SCA (mediante funciones de transferencia en S o V) para selección de características en conjuntos de datos médicos, con resultados competitivos en precisión de clasificación. Asimismo, se ha utilizado para optimizar los hiperparámetros de máquinas de soporte vectorial (SVR) y redes neuronales multicapa.

Segmentación de imágenes y diagnóstico médico. El SCA se ha aplicado a la segmentación de imágenes médicas (resonancia magnética, tomografía computarizada) mediante el enfoque de umbralización multinivel. También ha sido empleado en el diagnóstico de cáncer de pulmón y en el ajuste de parámetros de modelos fotovoltaicos.

Ingeniería mecánica y estructural. Entre las aplicaciones destacan el diseño óptimo de la sección transversal de perfiles aerodinámicos, la optimización de sistemas de engranajes, la detección de daños estructurales y el diseño de componentes de automóviles.

Robótica y telecomunicaciones. El SCA ha sido utilizado para la planificación de trayectorias de robots (incluyendo vehículos autónomos subacuáticos) y para la colocación óptima de cámaras en sistemas de vigilancia y redes de sensores inalámbricos.

8. Variantes y extensiones

La comunidad ha propuesto numerosas mejoras al SCA original para paliar sus limitaciones:

-) **SCA-PSO / ASCA-PSO**: hibridación con Particle Swarm Optimization para mejorar la explotación local.
-) **OBSCA**: incorpora aprendizaje por oposición (*opposition-based learning*) para aumentar la diversidad inicial.
-) **SCA caótico**: sustituye los parámetros aleatorios por mapas caóticos (logístico, de Chebyshev) para mejorar la cobertura del espacio.
-) **MO-SCA**: extensión multi-objetivo que mantiene un archivo de soluciones Pareto-óptimas.
-) **SCA binario**: adapta la actualización continua al espacio $\{0, 1\}^d$ para problemas de selección de características.
-) **EBSCA**: refuerza la fase de explotación mediante una ecuación de actualización centrada en el mejor individuo con un factor de equilibrio dinámico.

9. Conclusiones

El Sine Cosine Algorithm es una metaheurística elegante, de implementación sencilla y costo computacional reducido, que aprovecha las propiedades oscilantes y periódicas de las funciones trigonométricas para navegar el espacio de búsqueda. Su mecanismo de transición automática entre exploración y explotación, basado en el decaimiento lineal de r_1 , lo convierte en un punto de partida eficaz para una amplia gama de problemas continuos de optimización. Sin embargo, como dicta el teorema *No Free Lunch*, el SCA no es universalmente superior: en problemas altamente multimodales o de gran dimensión su convergencia puede estancarse en óptimos locales y su limitada memoria colectiva reduce la calidad de las soluciones. La rica familia de variantes híbridas y mejoradas surgidas desde 2016 refleja tanto el interés de la comunidad como el espacio de mejora existente sobre la versión original.

Referencias

- [1] S. Mirjalili, *SCA: A Sine Cosine Algorithm for solving optimization problems*, Knowledge-Based Systems, vol. 96, pp. 120–133, 2016. <https://doi.org/10.1016/j.knosys.2015.12.022>
- [2] L. Abualigah et al., *A comprehensive survey of sine cosine algorithm: variants and applications*, Artificial Intelligence Review, 2021. <https://doi.org/10.1007/s10462-021-10026-y>
- [3] M. A. Tawhid and K. B. Dsouza, *A comprehensive survey on the sine-cosine optimization algorithm*, ResearchGate preprint, 2022.

- [4] R. Sayed et al., *Binary Sine Cosine Algorithms for Feature Selection from Medical Data*, arXiv:1911.07805, 2019.
- [5] J. Liu et al., *An exploitation-boosted sine cosine algorithm for global optimization*, Engineering Applications of Artificial Intelligence, 2022.
- [6] X. Wang et al., *Improved Sine Cosine Algorithm for Optimization Problems Based on Self-Adaptive Weight and Social Strategy*, IEEE Access, 2023.